CRM Contact Sync - Authentication Report

Project: n8n CRM Contact Synchronization with OAuth2 Authorization Code Flow

1. OAuth2 Authorization Code Grant

The workflow is designed to authenticate CRM API requests using the OAuth2 Authorization Code grant. In n8n, the user starts the connection from an OAuth2 credential. The browser is redirected to the authorization server, the user grants permission, and the authorization server returns an authorization code to n8n's callback URL. n8n then exchanges that code at the token endpoint for an access token. The access token is used by the HTTP Request nodes as a Bearer token.

Setting	Configured value
Grant Type	Authorization Code
Authorization URL	https://auth.mockcrm.cloudsync.io/authorize
Token URL	https://auth.mockcrm.cloudsync.io/token
Client ID	crm_sync_4n8n
Scope	contacts.read contacts.write
Redirect URI	n8n-generated OAuth2 callback URL

The client secret was configured in n8n but is intentionally not reproduced in this report for security. The Search Contact, Create Contact, and Update Contact HTTP Request nodes are configured to use the OAuth2 credential.

2. Workflow Logic

The workflow starts with a POST Webhook that receives a contact object containing first_name, last_name, email, phone, title, department, and office. The contact data is validated, and the email value is used to search the CRM. If the search response contains an empty data array, the workflow sends the complete incoming contact object to the Create Contact endpoint. If an existing contact is returned, the workflow uses the first contact ID, constructs a full update object, and sends it to the Update Contact endpoint.

Expected successful create response:

```
{"status":"success","action":"created","contactId":"c_new1234"}
```

Expected successful update response:

```
{"status":"success","action":"updated","contactId":"c_8d7e6f5g"}
```

3. Error Handling

The HTTP Request nodes use n8n error outputs so that failures do not terminate silently. Search, Create, and Update errors are routed to a dedicated error-handling path. The workflow extracts the HTTP status and error message and responds to the webhook caller with a structured failure response.

The design covers client and server errors such as 400 validation failures, 404 not found errors, 409 duplicate-email errors, and 500 internal server errors. A typical failure response is:

```
{"status":"error","message":"CRM API request failed"}
```

4. Testing Performed

Test path	Expected result	Status
New contact	Search returns no contact; Create Contact runs; HTTP 201 response	Verified
Existing contact	Search returns an existing ID; Update Contact runs; HTTP 200 response	Verified
Simulated API error	Error branch runs and returns structured failure response	Evidence to attach

5. Challenge and Resolution

During OAuth2 connection testing, the supplied mock authorization endpoint returned an SSL certificate validation error (NET::ERR_CERT_AUTHORITY_INVALID). The OAuth2 credential was still configured using the authorization URL, token URL, client ID, scope, and Authorization Code grant specified in the assessment. Because the supplied mock OAuth service could not complete a trusted browser connection, temporary mock CRM endpoints were used only to verify workflow logic for search, create, update, and error paths. The final workflow configuration retains the assessment's required mockcrm.cloudsync.io API URLs and OAuth2 settings.

6. Conclusion

The CRM Contact Sync workflow implements the required search-to-create/update decision flow, sends complete JSON bodies, uses OAuth2 configuration in the HTTP Request nodes, and provides explicit success and failure webhook responses. Create and update paths were verified, and the error-handling path is designed to return failures without silent crashes.